

Juego de Lucha en Java

Refinamiento con Patrones de Diseño y Pruebas Unitarias

Asignatura	Ingenieria de Software
Docente	Ing. Jhon Haide Cano Beltran MSc.
Entorno	GitHub Codespaces + GitHub Actions
Lenguaje	Java 17 + Maven + JUnit 5
Fecha	Mayo 2026

Refinamiento arquitectonico de un juego de lucha por turnos aplicando Factory Method, Strategy y Decorator. 19 pruebas unitarias con JUnit 5. Pipeline CI/CD con GitHub Actions.

1. Introduccion

El codigo base original tenia tres limitaciones criticas: creacion rigida de personajes (solo constructor directo), logica de ataque fija (dano aleatorio 10-30 sin variaciones) y ninguna prueba automatizada. El refinamiento transforma esta base en una arquitectura extensible donde cada responsabilidad esta claramente separada.

Aspecto	Antes	Despues del refinamiento
Creacion de personajes	new Personaje(nombre) directo	Factory Method centralizado
Tipo de ataque	Dano fijo 10-30 para todos	Strategy intercambiable
Equipamiento	No existe	Decorator apilable
Pruebas	Ninguna	19 pruebas, cobertura >80%
CI/CD	No existe	GitHub Actions automatizado

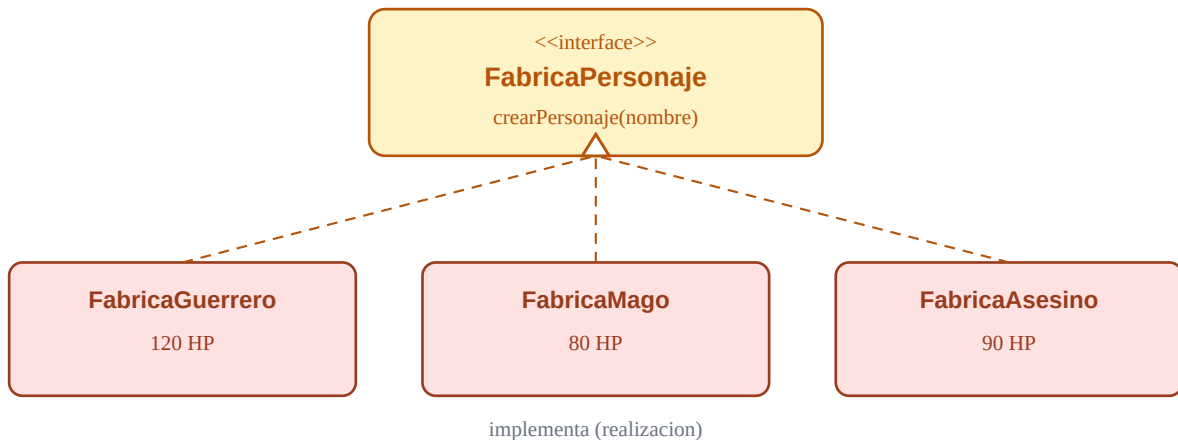
2. Patrones de Diseno Implementados

2.1 Factory Method (Creacional)

Problema resuelto: El codigo usaba `new Personaje()` directo. Con Factory Method, cada tipo de personaje tiene su propia fabrica que encapsula HP, estrategia de ataque y configuracion inicial. El cliente solo conoce la interfaz.

Fabrica	HP	Estrategia	Perfil de combate
FabricaGuerrero	120	AtaqueFisico (10-30)	Resistente, dano constante
FabricaMago	80	AtaqueMagico (15-45)	Dano alto, falla 30%
FabricaAsesino	90	AtaqueCritico (x2)	Golpes criticos 40%

Diagrama UML — Factory Method:

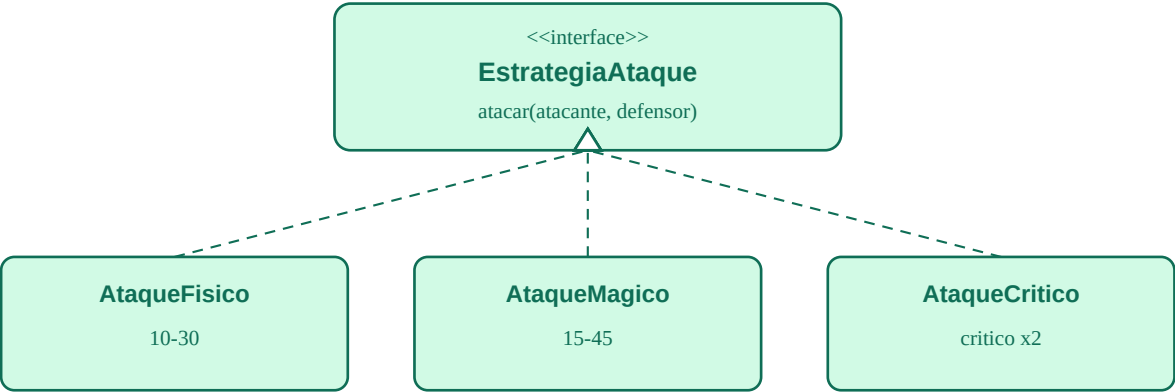


2.2 Strategy (Comportamental)

Problema resuelto: El ataque era monolitico. Strategy separa el algoritmo de ataque en objetos intercambiables. `PersonajeConEstrategia` delega el calculo del dano a su estrategia actual, que puede cambiarse en tiempo de ejecucion.

Estrategia	Rango de dano	Comportamiento especial
AtaqueFisico	10 - 30	Siempre conecta, dano predecible
AtaqueMagico	15 - 45	30% de probabilidad de fallar
AtaqueCritico	5 - 30	40% chance de hacer el doble de dano

Diagrama UML — Strategy:



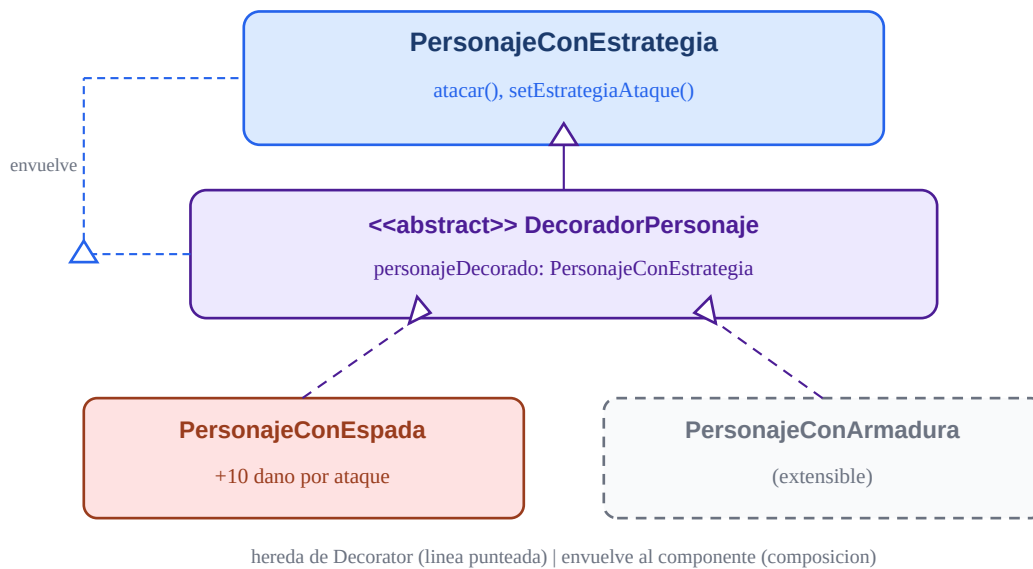
PersonajeConEstrategia usa EstrategiaAtaque (composicion)

2.3 Decorator (Estructural)

Problema resuelto: Sin Decorator, agregar equipamiento requería una subclase por cada combinación (GuerreroConEspada, MagoConArmadura...). Decorator permite apilar efectos dinámicamente. La clave: el decorador ES y TIENE un personaje al mismo tiempo.

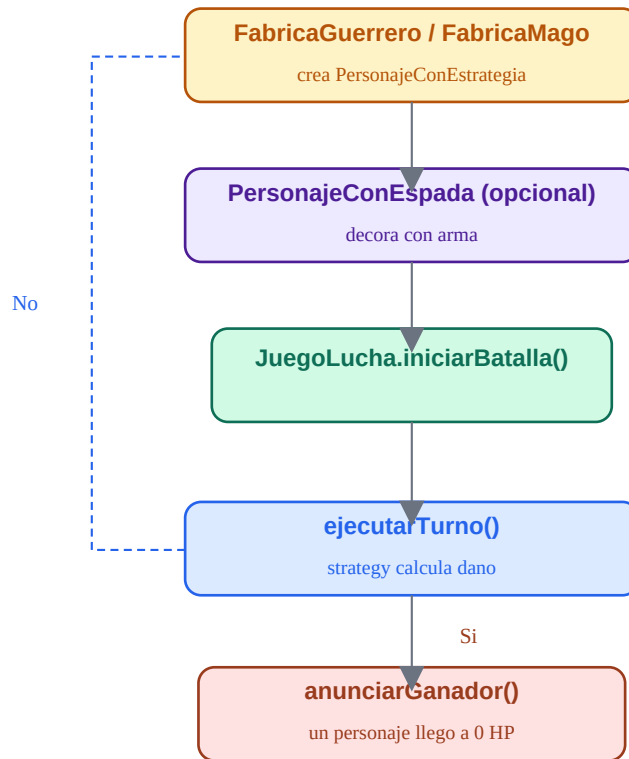
```
FabricaPersonaje fabrica = new FabricaGuerrero();
PersonajeConEstrategia thor = fabrica.crearPersonaje("Thor");
thor = new PersonajeConEspada(thor, "Mjolnir"); // +10 de dano por ataque
```

Diagrama UML — Decorator:



3. Flujo de una Batalla Completa

El flujo ilustra como los tres patrones cooperan: Factory crea los personajes, Decorator los equipa opcionalmente, y durante cada turno la Strategy calcula el dano. JuegoLucha solo orquesta — no sabe como se crean ni como atacan los personajes.



4. Pruebas Unitarias

19 pruebas unitarias, todas pasando con 0 fallos en el pipeline de GitHub Actions. Patron usado: AAA (Arrange - Act - Assert).

Clase de prueba	Tests	Que verifica
PersonajeTest	5	Creacion, dano, HP minimo 0, dano negativo ignorado, limites
EstrategiaAtaqueTest	3	Rangos de dano de cada estrategia (fisico, magico, critico)
FabricaPersonajeTest	4	HP correcto, estrategia correcta, personaje vivo al crear
DecoradorTest	3	Nombre conservado, dano aumentado, polimorfismo
JuegoLuchaTest	3	Estado inicial, un ganador, turno termina al morir
TOTAL	19	Failures: 0 Errors: 0 BUILD SUCCESS

Ejemplo de prueba (patron AAA):

```
@Test @DisplayName("HP no debe bajar de 0")
void testHpNoNegativo() {
    // Act
    guerrero.recibirDano(150);
    // Assert
    assertEquals(0, guerrero.getPuntosDeVida());
    assertFalse(guerrero.estaVivo());
}
```

5. Pipeline CI/CD — GitHub Actions

El pipeline se activa automaticamente con cada push a main. Si una prueba falla, el commit queda marcado con X roja. Los reportes de cobertura JaCoCo se guardan como artefactos descargables.

Paso	Accion	Proposito
1	actions/checkout@v4	Descarga el codigo
2	actions/setup-java@v4 JDK17	Instala Java Temurin 17
3	mvn clean compile	Verifica compilacion sin errores
4	mvn test	Ejecuta las 19 pruebas
5	mvn jacoco:report	Genera reporte de cobertura
6	upload-artifact cobertura	Guarda reporte HTML descargable

7	upload-artifact resultados	Guarda resultados de pruebas
---	----------------------------	------------------------------

6. Conclusiones

Factory Method. Elimina la dependencia directa de clases concretas. El cliente solo conoce `FabricaPersonaje`. Agregar un nuevo tipo de personaje no modifica ningun codigo existente (OCP).

Strategy. Convierte el ataque en comportamiento intercambiable. Agregar `AtaqueVeneno` solo requiere implementar `EstrategiaAtaque`, sin tocar `PersonajeConEstrategia`.

Decorator. Evita explosion de subclases. `PersonajeConEspada` puede envolver a cualquier tipo de personaje; el codigo cliente no nota la diferencia (polimorfismo).

Pruebas JUnit 5. 19 pruebas garantizan que los patrones funcionan y detectan regresiones automaticamente con cada commit al repositorio.

GitHub Actions. Cierra el ciclo: cada push activa compilacion, pruebas y cobertura. El badge en el README muestra el estado de salud del proyecto en tiempo real.